

4. React Router & SSR

1. 课程资料



react-router-demo.zip

853.17KB



nextjs 项目源码



nextjs-demo.zip

26.61MB



从零实现同构应用（SSR） 源码



heyi-ssr.zip

449.19KB



课程笔记源码：



react-router-demo.zip

906.16KB



heyi-ssr.zip

441.68KB



2. 课程目标



- 初级：
 - 掌握 react-router 基本使用
 - 了解 react-router 不同类型 history 的作用
 - 了解 ssr
- 中级：

- 掌握 react-router 所有 router 使用，及其运用场景
- 理解 react-router browser router 实现原理
- 掌握 ssr 核心概念，并能使用 nextjs
- 高级：
 - 掌握 react-router 实现原理
 - 深入理解 react-router 核心原理及源码
 - 理解 ssr 实现原理

3. 课程大纲

- react-router 详解
- 手写一个 react-router
 - 如何监听 url 的变化
 - 如何匹配 path，按什么规则
 - 如何渲染对应的组件
- SSR
 - SSR 方案的价值
 - Next.js 服务端渲染开发指南
 - 深入理解原理 hydrate 与 renderToString

4. react-router 详解



我们统一使用 react-router v6.15.0，不同版本可能存在差异，我们尽量选择一致的版本。

4.1 基础使用

首先初始化 react 项目

```
1 npx create-react-app react-router-demo --template typescript
```

首先安装 react-router-dom

```
1 {  
2   "react-router-dom": "6.15.0"  
3 }
```

首先我们记住一点，其实在 react-router 底层，实现了不同宿主的路由，react-router-dom 是针对浏览器的，还有 react-router-native 是针对 react-native 的，这个思路非常值得我们学习，**上层抽象，具体宿主环境实现**

其他可选择：@tanstack/router

React-router 提供给我们两种定义路由的方式（组件式、函数创建式），我们只关注一种，**函数创建式**。

我们推荐使用函数创建式，定义路由，原因有以下几点：

1. 动态路由与路由权限的处理更直观
2. ts 类型推导与类型安全更健全

4.1.1 定义 router

```
1 import {  
2   createBrowserRouter,  
3 } from "react-router-dom";  
4  
5 const router = createBrowserRouter([  
6   {  
7     path: "/",
```

```
8     element: <div>Hello world!</div>,
9   },
10  ]);
```

4.1.2 RouterProvider

通过 router provider 使用路由

```
1  import * as React from "react";
2  import * as ReactDOM from "react-dom/client";
3  import {
4    createBrowserRouter,
5    RouterProvider,
6  } from "react-router-dom";
7
8  const router = createBrowserRouter([
9    {
10     path: "/",
11     element: <div>Hello world!</div>,
12   },
13  ]);
14
15  ReactDOM.createRoot(document.getElementById("root")).render(
16    <React.StrictMode>
17      <RouterProvider router={router} />
18    </React.StrictMode>
19  );
```

这样一个简单带路由的应用就创建成功啦。

4.2 常见的几种 router

- createBrowserRouter: 浏览器提供的历史管理
- createHashRouter: 基于 hash 的路由管理, #hello, 但是呢通常 # 又可以作为锚链接
- createMemoryRouter: 内存型路由, 路由的管理存储在**内存**中
- createStaticRouter: SSR 服务端的

4.2.1 createBrowserRouter

通过浏览器原生路由进行路由态管理, 页面跳转通过 pushState、popState 方法实现

```
1  import * as React from "react";
```

```

2  import * as ReactDOM from "react-dom";
3  import {
4    createBrowserRouter,
5    RouterProvider,
6  } from "react-router-dom";
7
8  import Root, { rootLoader } from "../routes/root";
9  import Team, { teamLoader } from "../routes/team";
10
11 const router = createBrowserRouter([
12   {
13     path: "/",
14     element: <Root />,
15     loader: rootLoader,
16     children: [
17       {
18         path: "team",
19         element: <Team />,
20         loader: teamLoader,
21       },
22     ],
23   },
24 ]);
25
26 ReactDOM.createRoot(document.getElementById("root")).render(
27   <RouterProvider router={router} />
28 );

```



需要注意的是，使用 browserRouter，一般都需要使用类似 Nginx 做静态资源代理，另外需要注意 404 的情况，一般都需要添加 **try_files** 处理

Nginx try_files

```

1  location / {
2    try_files $uri /index.html;
3  }

```

4.2.2 createHashRouter

请注意，这个方法非常不推荐，所以我们不会重点讲，简单过一下。

他的用武之地就在于，我们没有 Nginx 作为静态资源代理，我们可能就无法使用浏览器历史作为我们路由状态的存储，这时可以选择 hash router 方案，但是注意，真的**非常不推荐**，除非是你自己的个人

项目。

```
1  import * as React from "react";
2  import * as ReactDOM from "react-dom";
3  import {
4    createHashRouter,
5    RouterProvider,
6  } from "react-router-dom";
7
8  import Root, { rootLoader } from "../routes/root";
9  import Team, { teamLoader } from "../routes/team";
10
11 const router = createHashRouter([
12   {
13     path: "/",
14     element: <Root />,
15     loader: rootLoader,
16     children: [
17       {
18         path: "team",
19         element: <Team />,
20         loader: teamLoader,
21       },
22     ],
23   },
24 ]);
25
26 ReactDOM.createRoot(document.getElementById("root")).render(
27   <RouterProvider router={router} />
28 );
```

4.2.3 createMemoryRouter

用于创建一个内存型路由，路由表与历史记录栈存储在内存中，当页面刷新时，路由信息丢失。

大家还记得我们在 Vue3 项目实战的那个例子吗？页面中有一个浏览器模拟器，他的路由我们则可以选择内存路由来处理。

```
1  import * as React from "react";
2  import * as ReactDOM from "react-dom";
3  import {
4    createMemoryRouter,
5    RouterProvider,
6  } from "react-router-dom";
7
```

```

8   import CalendarEvent from "../routes/calendarEvent";
9
10  const routes = [
11    {
12      path: "/events/:id",
13      element: <CalendarEvent />,
14      loader: () => FAKE_EVENT,
15    },
16  ];
17
18  const router = createMemoryRouter(routes, {
19    initialEntries: ["/", "/events/123"],
20    initialIndex: 1,
21  });
22
23  ReactDOM.createRoot(document.getElementById("root")).render(
24    <RouterProvider router={router} />
25  );

```

4.2.4 createStaticRouter

如果我们需要实现服务端渲染，那么在服务端的路由处理则需要使用该 api，因为我们知道客户端的路由是基于浏览器的 history，而服务端是没有浏览器环境的。

```

1   import {
2     createStaticHandler,
3     createStaticRouter,
4     StaticRouterProvider,
5   } from "react-router-dom/server";
6   import Root, {
7     loader as rootLoader,
8     ErrorBoundary as RootBoundary,
9   } from "../root";
10
11  const routes = [
12    {
13      path: "/",
14      loader: rootLoader,
15      Component: Root,
16      ErrorBoundary: RootBoundary,
17    },
18  ];
19
20  export async function renderHtml(req) {
21    let { query, dataRoutes } = createStaticHandler(routes);

```

```

22     let fetchRequest = createFetchRequest(req);
23     let context = await query(fetchRequest);
24
25     // If we got a redirect response, short circuit and let our Express server
    // handle that directly if (context instanceof Response) {
26         throw context;
27     }
28
29     let router = createStaticRouter(dataRoutes, context);
30     return ReactDOMServer.renderToString(
31         <React.StrictMode>
32             <StaticRouterProvider          router={router}          context={context}>
33                 />
34             </React.StrictMode>
35     );

```

4.3 常用的 Hook

- `useNavigate`
- `useLocation`
- `useMatch`

4.3.1 `useNavigate`

该 hook 让你可以在组件中进行跳转

```

1     import { useNavigate } from "react-router-dom";
2
3     function useLogoutTimer() {
4         const userIsInactive = useFakeInactiveUser();
5         const navigate = useNavigate();
6
7         useEffect(() => {
8             if (userIsInactive) {
9                 fake.logout();
10                navigate("/session-timed-out");
11            }
12        }, [userIsInactive]);
13    }

```

还有一个 `redirect` 方法，可以在组件外部需要重定向时（`loaders` 和 `actions` 中），使用

```
1  import { redirect } from "react-router-dom";
2
3  const loader = async () => {
4    const user = await getUser();
5    if (!user) {
6      return redirect("/login");
7    }
8    return null;
9  };
```

4.3.2 useLocation

用于获取当前 location 相关信息

4.3.3 useMatch

返回动态路由匹配到的值，例如现在路由定义：/person/:id

而此时地址栏路由为：/person/88

那么 useMatch 返回的值便是 88

4.4 常见组件

Link

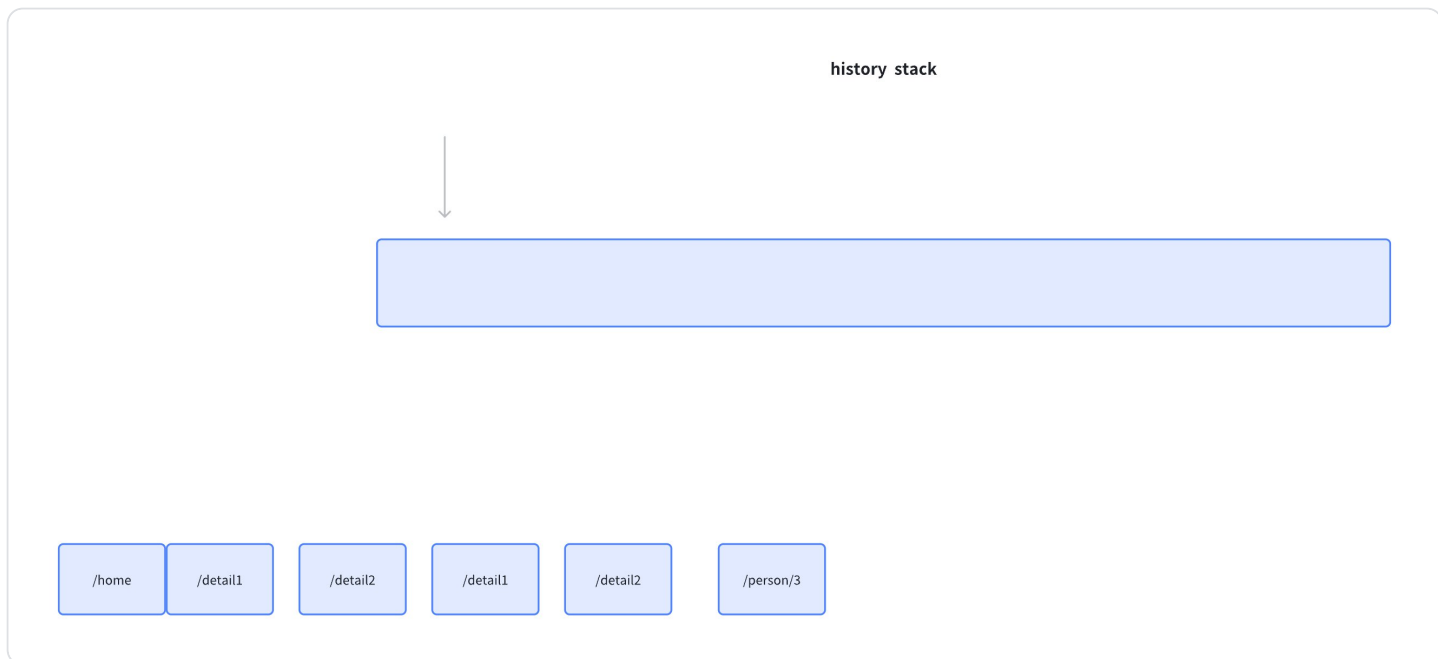
因为**语义**以及用户可用性

比如跳转用 onClick 实现，那么你就必须通过鼠标点击才能跳转，如果要在新的 tab 打开，如果是 Link，按住 Ctrl+左键即可，navigate 的话，就不行

4.5 原理浅析

分析源码我们需要重点关注：`react-router`、`react-router-dom` 和 `history`。

路由历史记录栈



竞争式路由

5. 手写一个 react-router

我们前面提到，react 给我们提供了四种不同的路由系统，分别为：

1. Browser
2. Hash
3. Memory
4. Static

通常工作中，大家使用 `BrowserRouter` 居多，面试也常问，我们就以 `BrowserRouter` 为例梳理其实现原理，理解后，大家可以融合贯通，这四种系统是怎样通过 `createxxxRouter` 抹平端差异的。

5.1 BrowserRouter

浏览器路由实现，最主要的两个概念是变更路由与监听路由变更。



History 是整个浏览器路由历史记录大对象，其中包含长度等属性

Location 是单一页面所对应的资源定位对象，我们可以理解为当页面跳转时，先生成 Location，然后将 Location push 到 History。

首先我们要明确几个概念：

1. 历史操作，`history.pushState`、`history.replaceState`。不会触发 **popstate**
2. 监听变更，`window.addEventListener("popstate", () => {})`
3. 操作，`history.back()`，`history.forward()`，`history.go()`，会触发 **popstate**

创建 router

```

1  export function createBrowserRouter(
2    routes: RouteObject[],
3    opts?: DOMRouterOpts
4  ): RemixRouter {
5    return createRouter({
6      basename: opts?.basename,
7      future: {
8        ...opts?.future,
9        v7_prependBasename: true,
10     },
11     history: createBrowserHistory({ window: opts?.window }),
12     hydrationData: opts?.hydrationData || parseHydrationData(),
13     routes,
14     mapRouteProperties,
15   }).initialize();
16 }

```

router 下承载 history

```

1  /**
2   * Browser history stores the location in regular URLs. This is the standard
   for
3   * most web apps, but it requires some configuration on the server to ensure
   you
4   * serve the same app at multiple URLs.
5   *
6   * @see https://github.com/remix-run/history/tree/main/docs/api-
   reference.md#createbrowserhistory
7   */
8  export function createBrowserHistory(
9    options: BrowserHistoryOptions = {}
10 ): BrowserHistory {
11    function createBrowserLocation(
12      window: Window,
13      globalHistory: Window["history"]
14    ) {
15      let { pathname, search, hash } = window.location;
16      return createLocation(
17        "",
18        { pathname, search, hash },
19        // state defaults to `null` because `window.history.state` does
20        (globalHistory.state && globalHistory.state.usr) || null,

```

```

21     (globalHistory.state && globalHistory.state.key) || "default"
22   );
23 }
24
25 function createBrowserHref(window: Window, to: To) {
26   return typeof to === "string" ? to : createPath(to);
27 }
28
29 return getUrlBasedHistory(
30   createBrowserLocation,
31   createBrowserHref,
32   null,
33   options
34 );
35 }

```

history 需要 location 定位资源

```

1  function getUrlBasedHistory(
2    getLocation: (window: Window, globalHistory: Window["history"]) => Location,
3    createHref: (window: Window, to: To) => string,
4    validateLocation: ((location: Location, to: To) => void) | null,
5    options: UrlHistoryOptions = {}
6  ): UrlHistory {
7    let { window = document.defaultView!, v5Compat = false } = options;
8    let globalHistory = window.history;
9    let action = Action.Pop;
10   let listener: Listener | null = null;
11
12   let index = getIndex()!;
13   // Index should only be null when we initialize. If not, it's because the
14   // user called history.pushState or history.replaceState directly, in which
15   // case we should log a warning as it will result in bugs.
16   if (index == null) {
17     index = 0;
18     globalHistory.replaceState({ ...globalHistory.state, idx: index }, "");
19   }
20
21   function getIndex(): number {
22     let state = globalHistory.state || { idx: null };
23     return state.idx;
24   }
25
26   function handlePop() {
27     action = Action.Pop;

```

```

28     let nextIndex = getIndex();
29     let delta = nextIndex == null ? null : nextIndex - index;
30     index = nextIndex;
31     if (listener) {
32         listener({ action, location: history.location, delta });
33     }
34 }
35
36 function push(to: To, state?: any) {
37     action = Action.Push;
38     let location = createLocation(history.location, to, state);
39     if (validateLocation) validateLocation(location, to);
40
41     index = getIndex() + 1;
42     let historyState = getHistoryState(location, index);
43     let url = history.createHref(location);
44
45     // try...catch because iOS limits us to 100 pushState calls :/
46     try {
47         globalHistory.pushState(historyState, "", url);
48     } catch (error) {
49         // If the exception is because `state` can't be serialized, let that
50         // outwards just like a replace call would so the dev knows the cause
51         // https://html.spec.whatwg.org/multipage/nav-history-apis.html#shared-
52         // history-push/replace-state-steps
53         // https://html.spec.whatwg.org/multipage/structured-
54         // data.html#structuredserializeinternal
55         if (error instanceof DOMException && error.name === "DataCloneError") {
56             throw error;
57         }
58         // They are going to lose state here, but there is no real
59         // way to warn them about it since the page will refresh...
60         window.location.assign(url);
61     }
62
63     if (v5Compat && listener) {
64         listener({ action, location: history.location, delta: 1 });
65     }
66 }
67
68 function replace(to: To, state?: any) {
69     action = Action.Replace;
70     let location = createLocation(history.location, to, state);
71     if (validateLocation) validateLocation(location, to);
72
73     index = getIndex();

```

```

72     let historyState = getHistoryState(location, index);
73     let url = history.createHref(location);
74     globalHistory.replaceState(historyState, "", url);
75
76     if (v5Compat && listener) {
77         listener({ action, location: history.location, delta: 0 });
78     }
79 }
80
81 function createURL(to: To): URL {
82     // window.location.origin is "null" (the literal string value) in Firefox
83     // under certain conditions, notably when serving from a local HTML file
84     // See https://bugzilla.mozilla.org/show_bug.cgi?id=878297
85     let base =
86         window.location.origin !== "null"
87             ? window.location.origin
88             : window.location.href;
89
90     let href = typeof to === "string" ? to : createPath(to);
91     invariant(
92         base,
93         `No window.location.(origin|href) available to create URL for href:
94     ${href}`
95     );
96     return new URL(href, base);
97 }
98
99 let history: History = {
100     get action() {
101         return action;
102     },
103     get location() {
104         return getLocation(window, globalHistory);
105     },
106     listen(fn: Listener) {
107         if (listener) {
108             throw new Error("A history only accepts one active listener");
109         }
110         window.addEventListener(PopStateEventType, handlePop);
111         listener = fn;
112
113         return () => {
114             window.removeEventListener(PopStateEventType, handlePop);
115             listener = null;
116         };
117     },
118     createHref(to) {

```

```

118     return createHref(window, to);
119 },
120 createURL,
121 encodeLocation(to) {
122     // Encode a Location the same way window.location would
123     let url = createURL(to);
124     return {
125         pathname: url.pathname,
126         search: url.search,
127         hash: url.hash,
128     };
129 },
130 push,
131 replace,
132 go(n) {
133     return globalHistory.go(n);
134 },
135 };
136
137 return history;
138 }

```

事件处理解决后，接下来就是解决监听，我们上面提到监听 popState 以此来处理路由变更

```

1 listen(fn: Listener) {
2     if (listener) {
3         throw new Error("A history only accepts one active listener");
4     }
5     window.addEventListener(PopStateEventType, handlePop);
6     listener = fn;
7
8     return () => {
9         window.removeEventListener(PopStateEventType, handlePop);
10        listener = null;
11    };
12 },

```

处理 popstate 逻辑

```

1 function handlePop() {
2     action = Action.Pop
3     let nextIndex = getIndex()
4     let delta = nextIndex == null ? null : nextIndex - index

```

```
5     index = nextIndex
6     if (listener) {
7         listener({ action, location: history.location, delta })
8     }
9 }
```

6. SSR

ssr 全称 server-side render，意为服务端渲染

6.1 SSR 方案的价值

SSR 方案最大的用途可以概括为 **SEO 友好**、**极大提升网页访问速度（特别是首屏加载）**。其实这项技术非常古老，还记得我刚入门开发时，写过 Java，那会儿 Java 还用 **servlet + jsp** 编写页面+服务逻辑。现在前端通用 SSR 方案只是将 Java 换成了 nodejs。

6.2 Nextjs 服务端开发指南

我们使用 nextjs 提供的脚手架初始化项目

```
1  npx create-next-app nextjs-blog
```

安装依赖

```
1  pnpm i
```

启动应用

```
1  pnpm dev
```

6.2.1 约定式路由

什么是约定式路由？就是项目中 pages 下的文件结构直接对应于路由系统，比如我们现在有一个路由 `/heyi`，那么在 `pages` 文件夹下，就有一个 `heyi.ts`。



大家想想，为什么 nextjs 将路由设计为约定式路由呢？

6.2.2 Link、Head、Layout

Link 组件类似 react-router 的 Link，需要注意的是一般推荐跳转相关逻辑尽量使用 Link，为什么呢？因为语义以及用户可用性。

```
1  <Link href="/">
2    <a>Back to home</a>
3  </Link>
```

6.2.3 getStaticProps

用于获取组件初始值

该方法在运行时自动处理参数，并将处理后返回值交给 React 组件进行渲染

```
1  export default function Home(props) { ... }
2
3  export async function getStaticProps() {
4    // Get external data from the file system, API, DB, etc.
5    const data = ...
6
7    // The value of the `props` key will be
8    // passed to the `Home` component
9    return {
10      props: ...
11    }
12  }
```

前端react使用 `ReactDOM.hydrateRoot`，后端react使用 `ReactDOM.renderToString`

6.3 深入理解 hydrate 与 renderToString、renderToReadableStream

前面我们通过示例看到，react ssr 的方案可以大致总结为：

服务端将 react 组件代码转为 html 字符串，返回给客户端，客户端获取到内容后，需要进行注水及事件处理。

6.3.1 renderToString

在服务端，调用 renderToString 方法后，会将 `React` 元素转化为初始 `HTML`。需要注意的是，此时只是进行简单的模板解析转换，数据请求等，时间的绑定与数据的消费均不发生在此刻。

我们提到的服务端数据请求并提供给客户端的过程，称之为：**注水**，客户端拿到数据进行消费，称之为：**脱水**

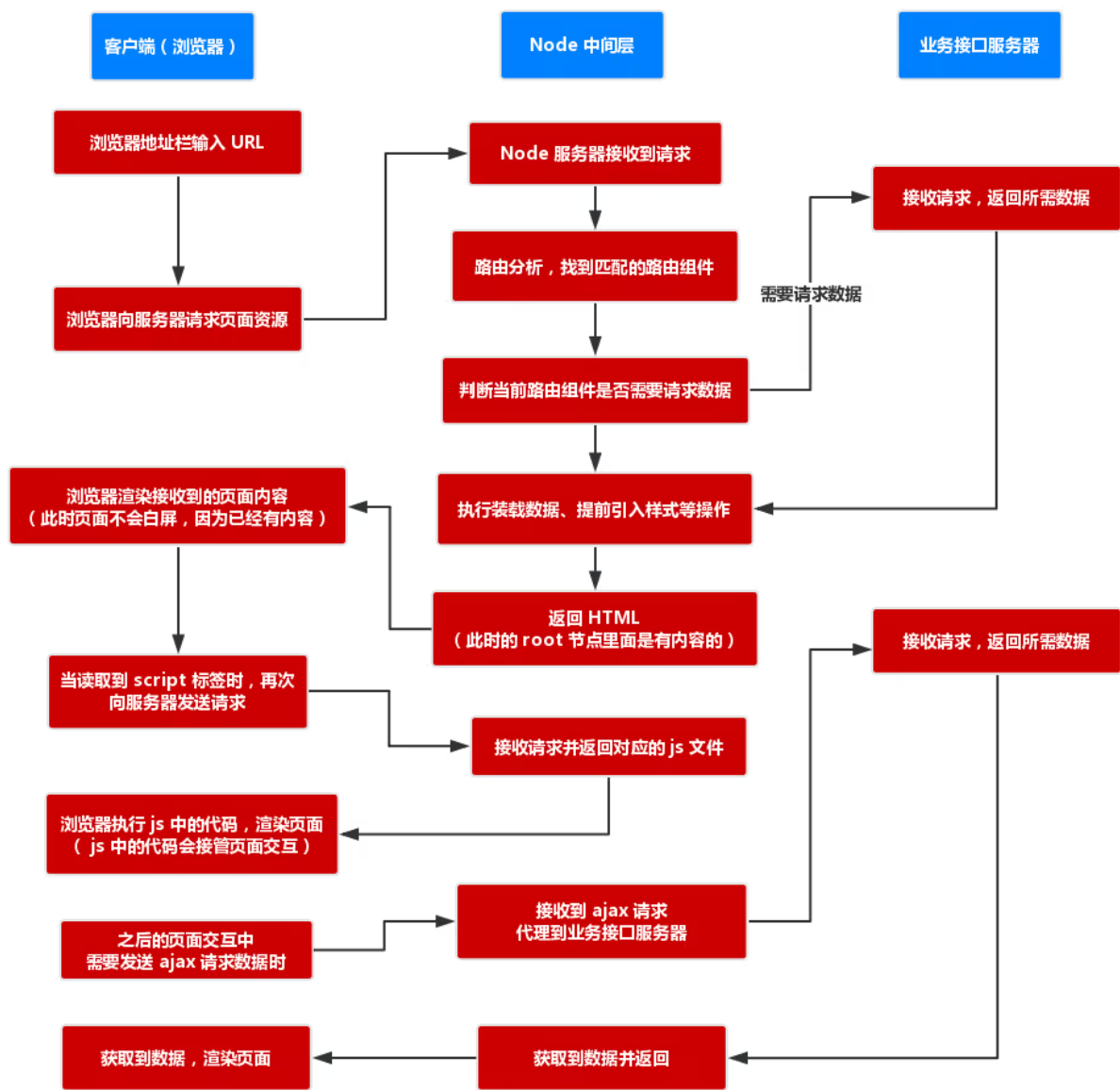
6.3.2 renderToReadableStream

流式 SSR，其实这也不是什么新概念，相信大家以前做过文件下载吧？很多时候我们文件下载的形式就是基于流。通过二进制流替代字符串类型返回。

6.3.3 hydrate

顾名思义，水合。

上面我们提到服务端将 react 内容转为 html string 并返回了，此时客户端需要请求组件数据并将数据渲染到页面，称为：**脱水**，另外，事件的绑定也发生在此刻，react 通过事件委托机制，实现组件的事件绑定，最终完成页面渲染。



具体实现见源码

```
1 node api/index.js
```

```
1 pnpm dev
```

6.3.4 注意点

1. `renderToString` 有什么作用？
2. 为什么服务端加载了一次，客户端还需要再次加载呢？
3. 服务端加载了 `React` 输出的代码片段，客户端又执行了一次，这样是不是会加载两次导致资源浪费呢？

6.3.5 ReactDOMServer.renderToString(element)

将 `React` 元素渲染为初始 `HTML`。`React` 将返回一个 `HTML` 字符串。你可以使用此方法在服务端生成 `HTML`，并在首次请求时将标记下发，以加快页面加载速度，并允许搜索引擎爬取你的页面以达到 `SEO` 优化的目的。

为什么服务端加载了一次，客户端还需要再次加载呢？

原因很简单，服务端使用 `renderToString` 渲染页面，而 `react-dom/server` 下的 `renderToString` 并没有做事件相关的处理，因此返回给浏览器的内容不会有事件绑定，渲染出来的页面只是一个静态的 `HTML` 页面。只有在客户端渲染 `React` 组件并初始化 `React` 实例后，才能更新组件的 `state` 和 `props`，初始化 `React` 的事件系统，让 `React` 组件真正“动”起来。

是否会加载两次？

如果你在已有服务端渲染标记的节点上调用 `ReactDOM.hydrate()` 方法，`React` 将会保留该节点且只进行事件处理绑定，从而让你有一个非常高性能的首次加载体验。因此不必担心加载多次的问题。

7. 手写同构应用

见源码

